

NumPy, which stands for Numerical Python, is a library consisting of multidimensional array objects and a collection of routines for processing those arrays. Using NumPy, mathematical and logical operations on arrays can be performed. Travis Oliphant created NumPy package in 2005 by injecting the features of the ancestor module Numeric into another module Numarray.

It is an extension module of Python which is mostly written in C. It provides various functions which are capable of performing the numeric computations with a high speed.

NumPy provides various powerful data structures, implementing multi-dimensional arrays and matrices. These data structures are used for the optimal computations regarding arrays and matrices.

The Need of NumPy

With the revolution of data science, data analysis libraries like NumPy, SciPy, Pandas, etc. have seen a lot of growth. With a much easier syntax than other programming languages, python is the first choice language for the data scientist.

NumPy provides a convenient and efficient way to handle the vast amount of data. NumPy is also very convenient with Matrix multiplication and data reshaping. NumPy is fast which makes it reasonable to work with a large set of data.

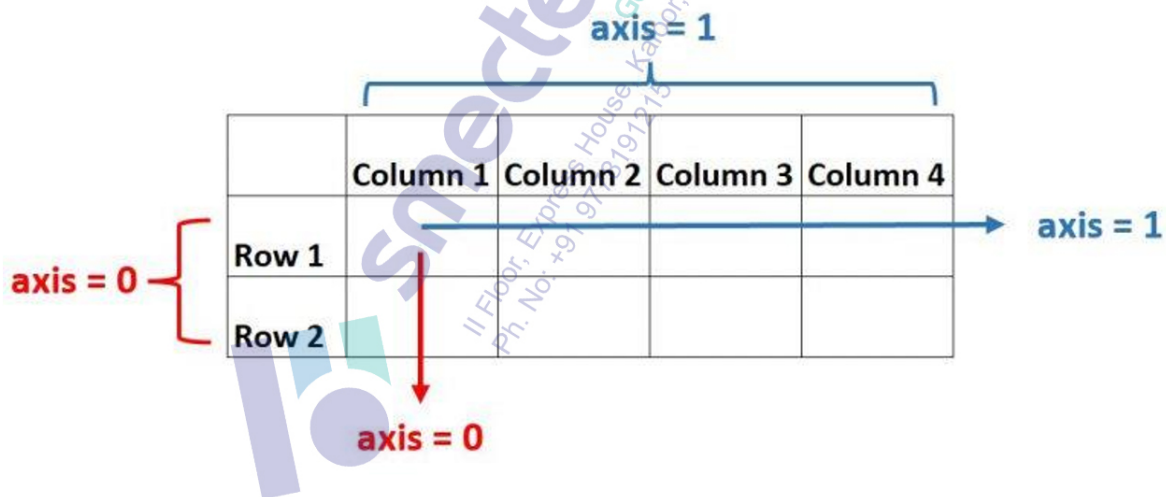
There are the following advantages of using NumPy for data analysis.

- Mostly written in C program
- No loops and statements (Process done in different method)

- NumPy performs array-oriented computing (Vectorized operation)
- It efficiently implements the multidimensional arrays.
- It performs scientific computations.
- It is capable of performing Fourier Transform and reshaping the data stored in multidimensional arrays.
- NumPy provides the in-built functions for linear algebra and random number generation.

Ndarray Object

The most important object defined in NumPy is an N-dimensional array type called ndarray. It describes the collection of items of the same type. Items in the collection can be accessed using a zero-based index. An instance of ndarray class can be constructed by different array creation routines described later.



The basic ndarray is created using an array function in NumPy which is as follows :

1. numpy.array

It creates an ndarray from any object exposing array interface, or from any method that returns an array.

Syntax:

```
numpy.array(object, dtype = None, copy = True, order = None,
subok = False, ndmin = 0)
```

- *object* : Any object exposing the array interface method returns an array, or any (nested) sequence.
- *dtype* : Desired data type of array, optional
- *copy* : Optional. By default (true), the object is copied
- *order* : C (row major) or F (column major) or A (any) (default)
- *subok* : By default, returned array forced to be a base class array. If true, sub-classes passed through
- *ndmin* : Specifies minimum dimensions of resultant array.

Create a 1 dimension array(Vector)

```
import numpy as np
a = np.array([1,2,3])
print(a)
```

output:

```
[1 2 3]
```

Create a 2 dimension array (Matrix)

```
import numpy as np
a = np.array([[1, 2], [3, 4]])
print(a)
```

output:

```
[[1 2]
 [3 4]]
```

Data Types

NumPy supports a much greater variety of numerical types than Python does. The following table shows different scalar data types defined in NumPy.

1. `bool_` : Boolean (True or False) stored as a byte
2. `int_` : Default integer type (same as C long; normally either int64 or int32)
3. `intc` : Default integer type (same as C long; normally either int64 or int32)
4. `intp` : Integer used for indexing (same as C ssize_t; normally either int32 or int64)
5. `int8` : Byte (-128 to 127)
6. `int16` : Integer (-32768 to 32767)
7. `int32` : Integer (-2147483648 to 2147483647)

8. `int64` : Integer (-9223372036854775808 to 9223372036854775807)
9. `uint8` : Unsigned integer (0 to 255)
10. `uint16` : Unsigned integer (0 to 65535)
11. `uint32` : Unsigned integer (0 to 4294967295)
12. `uint64` : Unsigned integer (0 to 18446744073709551615)
13. `float_` : Shorthand for float64
14. `float16` : Half precision float: sign bit, 5 bits exponent, 10 bits mantissa
15. `float32` : Single precision float: sign bit, 8 bits exponent, 23 bits mantissa
16. `float64` : Double precision float: sign bit, 11 bits exponent, 52 bits mantissa

Assigning datatype to array

```
import numpy as np
a = np.array([1, 2, 3], dtype = complex)
print(a)
```

output:

```
[1.+0.j 2.+0.j 3.+0.j]
```

Assigning datatype to array using array-scalar type

```
import numpy as np
dt = np.dtype(np.int32)
```

```
print(dt)
```

output:

int32

Note

int8, int16, int32, int64 can be replaced by equivalent string 'i1', 'i2', 'i4', etc.

```
import numpy as np
dt = np.dtype('i4')
print(dt)
```

output:

int32

Use of structured data type. Here, the field name and the corresponding scalar data type is to be declared.

first create structured data type

```
import numpy as np
dt = np.dtype([('age', np.int8)])
print(dt)
```

output:

[('age', 'i1')]

now apply it to ndarray object

```
import numpy as np
dt = np.dtype([('age', np.int8)])
a = np.array([(10,), (20,), (30,)], dtype = dt)
print(a)
```

output:

```
[(10,) (20,) (30,)]
```

field name can be used to access content of age column

```
import numpy as np
dt = np.dtype([('age', np.int8)])
a = np.array([(10,), (20,), (30,)], dtype = dt)
print(a['age'])
```

output:

```
[10 20 30]
```

Define a structured data type called student with a string field 'name', an integer field 'age' and a float field 'marks'. This dtype is applied to ndarray object.

first create structured data type

```
import numpy as np
student = np.dtype([('name', 'S20'), ('age', 'i1'), ('marks', 'f4')])
print(student)
```

output:

```
[('name', 'S20'), ('age', 'i1'), ('marks', '<f4')]
```

now apply it to ndarray object

```
import numpy as np
student = np.dtype([('name', 'S20'), ('age', 'i1'), ('marks', 'f4')])
a = np.array([('abc', 21, 50), ('xyz', 18, 75)], dtype = 'student')
print(a)
```

output:

```
[(b'abc', 21, 50.) (b'xyz', 18, 75.)]
```

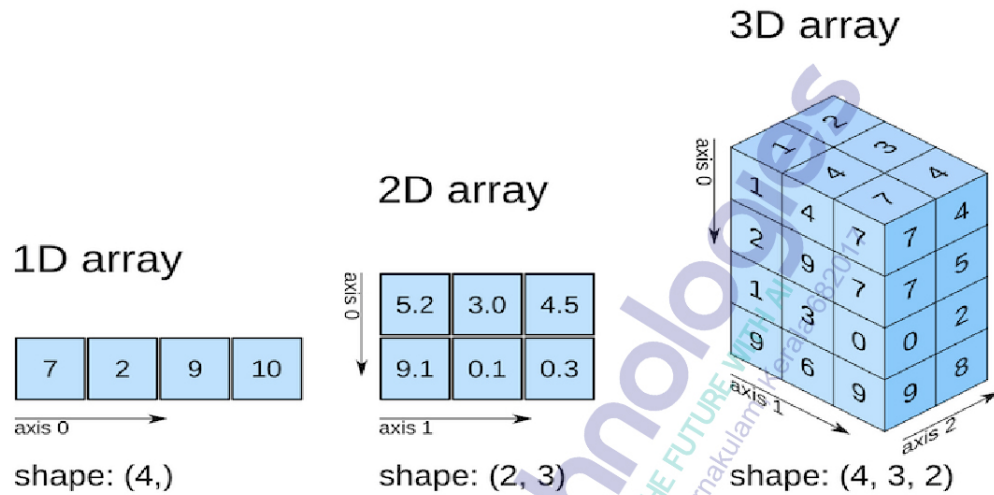
Each built-in data type has a character code that uniquely identifies it.

'b'	–	Boolean
'i'	–	(signed) integer
'u'	–	unsigned integer
'f'	–	floating-point
'c'	–	complex-floating point
'm'	–	timedelta
'M'	–	datetime
'O'	–	(Python) objects
'S', 'a'	–	(byte-)string
'U'	–	Unicode
'V'	–	raw data (void)

Array Attributes

1. ndarray.shape

This array attribute returns a tuple consisting of array dimensions. It can also be used to resize the array.



```
import numpy as np
```

```
a = np.array([[1,2,3],[4,5,6]])
```

```
print(a)
```

```
print(a.shape)
```

output:

```
[[1 2 3]
```

```
 [4 5 6]]
```

```
(2, 3)
```

2. ndarray.ndim

This array attribute returns the number of array dimensions.

an array of evenly spaced numbers

```
import numpy as np
a=np.array([[1,2,3],[4,5,6]])
print(a)
print(a.ndim)
```

output:

```
[[1 2 3]
 [4 5 6]]
2
```

3. numpy.itemsize

This array attribute returns the length of each element of array in bytes.

```
# dtype of array is int8
import numpy as np
x = np.array([1,2,3,4,5], dtype = np.int8)
print(x)
print(x.itemsize)
```

output:

```
[1 2 3 4 5]
1
```

```
# dtype of array is now float32 (4 bytes)
import numpy as np
x = np.array([1,2,3,4,5], dtype = np.float32)
print(x)
print(x.itemsize)
```

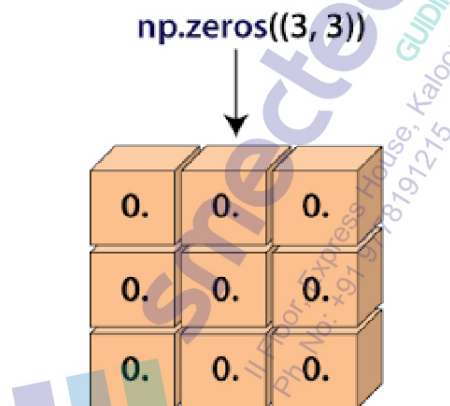
output:

```
[1.  2.  3.  4.  5.]
4
```

Array Creation Routines

1. `numpy.zeros`

Returns a new array of specified size, filled with zeros.



Syntax:

```
numpy.zeros(shape, dtype = float, order = 'C')
```

- `shape` - Shape of an empty array in int or sequence of int
- `dtype` - Desired output data type (Optional)

- *order* - 'C' for C-style row-major array, 'F' for FORTRAN style column-major array

array of five zeros. Default dtype is float

```
import numpy as np
x = np.zeros(5)
print(x)
```

output:

```
[0. 0. 0. 0. 0.]
```

custom type

```
import numpy as np
x = np.zeros([2,2], dtype='int8')
print(x)
```

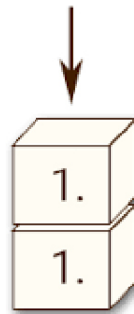
output:

```
[[0 0]
 [0 0]]
```

2. **numpy.ones**

Returns a new array of specified size and type, filled with ones.

`np.ones(( 2, 1 ))`



Syntax:

```
numpy.ones(shape, dtype = None, order = 'C')
```

- *shape* - Shape of an empty array in int or tuple of int.
- *dtype* - Desired output data type (Optional).
- *order* - 'C' for C-style row-major array, 'F' for FORTRAN style column-major array.

array of five ones. Default dtype is float

```
import numpy as np
```

```
x = np.ones(5)
```

```
print(x)
```

output:

```
[1. 1. 1. 1. 1.]
```

matrix of ones.

```
import numpy as np
```

```
x = np.ones([2,2], dtype = int)
print(x)
```

output:

```
[[1 1]
 [1 1]]
```

3. numpy.fromiter

This function builds an ndarray object from any iterable object. A new one-dimensional array is returned by this function.

Syntax:

```
numpy.fromiter(iterable, dtype, count = -1)
```

- *iterable* - Any iterable object
- *dtype* - Data type of resultant array
- *count* - The number of items to be read from iterator. Default is -1 which means all data to be read

Obtain iterator object from list

```
import numpy as np
mylist = range(5)
it = iter(mylist)
```

use iterator to create ndarray

```
x = np.fromiter(it, dtype = float)
print(x)
```

output:

```
[0. 1. 2. 3. 4.]
```

Array from Numerical Ranges

1. `numpy.arange`

This function returns an ndarray object containing evenly spaced values within a given range. The format of the function is as follows :

Syntax:

```
numpy.arange(start, stop, step, dtype)
```

- `start` : The start of an interval. If omitted, defaults to 0
- `stop` : The end of an interval (not including this number)
- `step` : Spacing between values, default is 1
- `dtype` : Data type of resulting ndarray. If not given, data type of input is used

```
import numpy as np
x = np.arange(5)
print(x)
```

output:

```
[0 1 2 3 4]
```

```
import numpy as np
```

```
# dtype set
x = np.arange(5, dtype = float)
print(x)
```

output:

```
[0. 1. 2. 3. 4.]
```

```
# start and stop parameters set
import numpy as np
x = np.arange(10,20,2)
print(x)
```

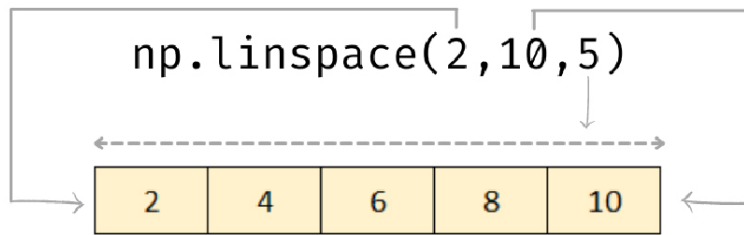
output:

```
[10 12 14 16 18]
```

2. numpy.linspace

This function is similar to `arange()` function. In this function, instead of step size, the number of evenly spaced values between the interval is specified.





Equally spaced values

Syntax:

```
numpy.linspace(start, stop, num, endpoint, retstep, dtype)
```

- **start** : The starting value of the sequence
- **stop** : The end value of the sequence, included in the sequence if endpoint set to true
- **num** : The number of evenly spaced samples to be generated.
Default is 50
- **endpoint** : True by default, hence the stop value is included in the sequence. If false, it is not included
- **retstep** : If true, returns samples and step between the consecutive numbers
- **dtype** : Data type of output ndarray

```
import numpy as np
x = np.linspace(10,20,5)
print(x)
```

output:

```
[10.  12.5 15.  17.5 20. ]
```

endpoint set to false

```
import numpy as np
```

```
x = np.linspace(10,20,5, endpoint = False)
```

```
print(x)
```

output:

```
[10. 12. 14. 16. 18.]
```

find retstep value

```
import numpy as np
```

```
x = np.linspace(1,2,5, retstep = True)
```

```
print(x)
```

output:

```
(array([1. , 1.25, 1.5 , 1.75, 2.  ]), np.float64(0.25))
```

Indexing & Slicing

Contents of ndarray object can be accessed and modified by indexing or slicing, just like Python's in-built container objects. Basic slicing is an extension of Python's basic concept of slicing to n dimensions. A Python slice object is constructed by giving start, stop, and step parameters to the built-

in slice function. This slice object is passed to the array to extract a part of array.

```
import numpy as np
a = np.arange(10)
print(a)
s = slice(2,7,2)
print(a[s])
```

output:

```
[0 1 2 3 4 5 6 7 8 9]
```

```
[2 4 6]
```

Slicing parameters separated by a colon : (start:stop:step) directly to the ndarray object.

data			data[0,1]			data[1:3]			data[0:2,0]		
	0	1		0	1		0	1		0	1
0	1	2		1	2		1	2		1	2
1	3	4		3	4		3	4		3	4
2	5	6		5	6		5	6		5	6

```
import numpy as np
a = np.arange(10)
print(a)
```

```
b = a[2:7:2]
print(b)
```

output:

```
[0 1 2 3 4 5 6 7 8 9]
[2 4 6]
```

```
# slice single item
import numpy as np
a = np.arange(10)
print(a)
b = a[5]
print(b)
```

output:

```
[0 1 2 3 4 5 6 7 8 9]
5
```

```
# slice items between indexes
import numpy as np
a = np.arange(10)
print(a)
print(a[2:5])
```

output:

```
[0 1 2 3 4 5 6 7 8 9]
```

```
[2 3 4]
```

```
import numpy as np
a = np.array([[1,2,3],[3,4,5],[4,5,6]])
print(a)
# slice items starting from index
print('Now we will slice the array from the index a[1:]')
print(a[1:])
```

output:

```
[[1 2 3]
 [3 4 5]
 [4 5 6]]
Now we will slice the array from the index a[1:]
[[3 4 5]
 [4 5 6]]
```

Slicing can also include ellipsis (...) to make a selection tuple of the same length as the dimension of an array. If ellipsis is used at the row position, it will return an ndarray comprising of items in rows.

```
# array to begin with
import numpy as np
a = np.array([[1,2,3],[3,4,5],[4,5,6]])
print('Our array is:')
```

```
print(a)
```

```
# this returns array of items in the second column
```

```
print('The items in the second column are:')
```

```
print(a[..., 1])
```

```
print('\n')
```

```
# Now we will slice all items from the second row
```

```
print('The items in the second row are:')
```

```
print(a[1, ...])
```

```
print('\n')
```

```
# Now we will slice all items from column 2 onwards
```

```
print('The items column 2 onwards are:')
```

```
print (a[...,1:])
```

output:

Our array is:

```
[[1 2 3]
```

```
 [3 4 5]
```

```
 [4 5 6]]
```

The items in the second column are:

```
[2 4 5]
```

The items in the second row are:

```
[3 4 5]
```

The items column 2 onwards are:

```
[[2 3]
 [4 5]
 [5 6]]
```

Advanced Indexing

There are two types of advanced indexing – Integer and Boolean.

Integer Indexing

This mechanism helps in selecting any arbitrary item in an array based on its Ndimensional index. Each integer array represents the number of indexes into that dimension. When the index consists of as many integer arrays as the dimensions of the target ndarray, it becomes straightforward.

```
import numpy as np
arr1 = np.array([[1, 2], [3, 4], [5, 6]])
print(arr1)
y = arr1[[0,1,2], [0,1,0]]
print(y)
```

output:

```
[[1 2]
 [3 4]
 [5 6]]
[1 4 5]
```

The selection includes elements at (0,0), (1,1) and (2,0) from the first array.

```
import numpy as np
x = np.array([[0, 1, 2], [3, 4, 5], [6, 7, 8], [9, 10, 11]])
print('Our array is:')
print(x)
print('\n')

rows = np.array([[0, 0], [3, 3]])
cols = np.array([[0, 2], [0, 2]])
y = x[rows, cols]

print('The corner elements of this array are:')
print(y)
```

output:

Our array is:

```
[[ 0  1  2]
 [ 3  4  5]
 [ 6  7  8]
 [ 9 10 11]]
```

The corner elements of this array are:

```
[[ 0  2]
 [ 9 11]]
```

Boolean Array Indexing

This type of advanced indexing is used when the resultant object is meant to be the result of Boolean operations, such as comparison operators. Items greater than 5 are returned as a result of Boolean indexing.

```
import numpy as np
x = np.array([[ 0,  1,  2],[ 3,  4,  5],[ 6,  7,  8],[ 9, 10, 11]])
print('Our array is:')
print(x)
print('\n')

# Now we will print the items greater than 5
print('The items greater than 5 are:')
print(x[x > 5])
```

output:

Our array is:

```
[[ 0  1  2]
 [ 3  4  5]
 [ 6  7  8]
 [ 9 10 11]]
```

The items greater than 5 are:

[6 7 8 9 10 11]

